

Curso: Técnicas de Programación

DOCUMENTACIÓN TÉCNICA FINAL

Proyecto #1 – Vitta

Sistema de gestión nutricional desarrollado en C# con Windows Forms, basado en arquitectura MVC y persistencia local en archivos CSV.

Estudiante	Natalia Tobal D.
Profesor	Luis Felipe Mora Umaña.
Fecha de entrega	07 de abril de 2026
Repositorio	https://github.com/Nat-92CR/ProyectoVitta.git
Gestión del proyecto	https://trello.com/b/3z5MMCwt/proyecto-sistema-de-gestion-nutricional-vitta
Versión documentada	Proyecto Vitta - Iteración 1

Documento elaborado con base en las instrucciones del proyecto, la explicación brindada en clase y la estructura real del repositorio entregado.

Índice

1.	Introducción	3
2.	Descripción general del sistema.....	4
3.	Alcance funcional y requisitos cubiertos	5
4.	Decisiones de diseño	6
5.	Technology Stack y entorno de ejecución	8
6.	Arquitectura, patrón aplicado y estructura del proyecto	8
7.	Modelo de datos y persistencia en archivos CSV	10
8.	Desarrollo del sistema y explicación de módulos	11
9.	Manual de instalación, ejecución e interacciones	13
10.	Resultados obtenidos y validación del proyecto	15
11.	Aprendizajes del desarrollo y Conclusiones	17
12.	Anexos Bibliografía	18

La organización del presente documento responde a lo solicitado en el enunciado del proyecto: portada, índice, introducción, decisiones de diseño, desarrollo, explicación de uso e interacciones, análisis de resultados, aprendizajes y conclusiones.

1. Introducción

Vitta es una aplicación de escritorio orientada al registro y control de la alimentación diaria. El proyecto fue desarrollado como primera iteración del curso Técnicas de Programación, por lo que el enfoque principal estuvo en construir una base extensible, comprensible y organizada, capaz de crecer hacia una segunda fase sin tener que rehacer toda la lógica del sistema.

El problema planteado por el proyecto consiste en permitir que un usuario se registre, gestione alimentos, construya menús diarios, consulte información nutricional y visualice estadísticas por fecha. Para responder a ese planteamiento se utilizó C# con Windows Forms en la capa visual, una separación por proyectos siguiendo arquitectura Modelo-Vista-Controlador y una persistencia simple basada en archivos CSV.

Además del funcionamiento, la entrega exige evidencia de buenas prácticas de desarrollo. Por esa razón el proyecto fue estructurado en tres capas, se utilizaron interfaces para desacoplar responsabilidades, se centralizó el acceso a datos y se incorporó análisis estático con SonarAnalyzer en el proyecto de la interfaz.

2. Descripción general del sistema

El sistema Vitta se concibe como una aplicación de escritorio orientada a la gestión nutricional individual. Su propósito es permitir que cada usuario registre información personal relevante, administre alimentos, construya menús diarios, consulte indicadores nutricionales y visualice estadísticas de seguimiento por fecha.

Para comprender el sistema de manera integral, a continuación, se presenta una descripción resumida de sus módulos principales, el propósito de cada uno y el resultado esperado dentro del funcionamiento general de la aplicación.

Componente	Propósito	Resultado esperado
Usuarios	Registrar nuevos usuarios, iniciar sesión y actualizar el perfil nutricional.	Disponer de información base para cálculos y personalización.
Alimentos	Crear, consultar, buscar, editar y eliminar alimentos con sus macronutrientes.	Construir un catálogo reutilizable para la elaboración de menús.
Menús	Registrar el consumo diario por tiempos de comida y mantenerlo por fecha.	Representar de manera clara qué comió el usuario cada día.
Información nutricional	Calcular calorías de mantenimiento, IMC y distribución recomendada de macros.	Ofrecer una referencia inmediata para la toma de decisiones.
Estadísticas	Comparar consumo vs. meta y resumir el comportamiento por fecha o período.	Transformar registros diarios en información útil y medible.

Desde el punto de vista de uso, la experiencia del usuario inicia en la pantalla de inicio de sesión. Después del acceso, la aplicación dirige al Dashboard, desde donde se habilitan los demás módulos del sistema.

3. Alcance funcional y requisitos cubiertos

El enunciado del proyecto establece como requisitos principales la gestión de usuarios, administración de alimentos y menús, una pestaña de información nutricional con cálculos base y un módulo de estadísticas parametrizable por fechas. Asimismo, solicita el uso de Windows Forms para las vistas, arquitectura MVC, persistencia local mediante archivos y aplicación de buenas prácticas de desarrollo.

Con el fin de evidenciar el cumplimiento de esos lineamientos, a continuación, se presenta una relación entre cada requisito solicitado, su implementación dentro de Vitta, la evidencia técnica correspondiente y el estado de cumplimiento alcanzado en esta primera iteración.

Requisito del Proyecto		Implementación en Vitta	Evidencia técnica	Estado
Gestión de usuarios	de	Registro, inicio de sesión y edición de perfil	RegisterView LoginView ProfileView	Evidenciado
Gestión de alimentos	de	Alta, consulta, búsqueda, edición y eliminación	FoodRegisterView FoodConsultView	Evidenciado
Gestión de menús		Registro, carga por fecha, modificación y eliminación	MenuRegisterView MenuController	Evidenciado
Información nutricional		Calorías de mantenimiento, IMC y macros	NutritionInfoView NutritionInfoController	Evidenciado
Estadísticas por fecha	por	Consumo diario, metas y consultas por período	StatisticsView StatisticsController	Evidenciado
Persistencia simple para iteración 1	para	Carga y guardado en archivos CSV	File Handler<T> ConfigurationItems	Evidenciado
Arquitectura MVC y separación por capas	y por	Organización del sistema en proyectos independientes para modelo, controlador y vista, con responsabilidades diferenciadas.	VittaModel, VittaController VittaView	Evidenciado
Conjunto de datos para la iteración 1	de	Uso de archivos con usuarios, alimentos y menús para pruebas, consultas y visualización por fechas.	users.csv foods.csv menus.csv	Evidenciado
Buenas prácticas y revisión de código		Uso de nombres descriptivos, separación de responsabilidades y SonarAnalyzer	Proyectos por capa e integración del paquete analizador	Evidenciado

4. Decisiones de diseño

Durante el desarrollo de Vitta no solo se buscó que el sistema cumpliera con las funciones solicitadas, sino también que tuviera una estructura ordenada, comprensible y fácil de trabajar. Por eso, a lo largo de la implementación se tomaron distintas decisiones de diseño relacionadas con la organización del proyecto, el manejo de los datos, la distribución de responsabilidades y la forma en que el usuario interactúa con el sistema.

Estas decisiones fueron importantes porque permitieron construir una solución más clara, más mantenible y coherente con lo solicitado en el proyecto. Además, ayudan a entender por qué el sistema fue desarrollado de esta manera y qué propósito cumple cada parte dentro del funcionamiento general de Vitta.

- **Separación por proyectos:**

La solución se dividió en VittaModel, VittaController y VittaView con el propósito de mantener responsabilidades claramente diferenciadas. Esta decisión evita mezclar la lógica de negocio, la gestión de datos y la interfaz gráfica dentro de un mismo espacio de trabajo. De esta manera, el modelo concentra las entidades principales del sistema, la capa de control coordina el comportamiento y la vista se limita a la interacción con el usuario. Esta organización hace que el proyecto resulte más comprensible, más ordenado y sencillo de revisar.

- **Uso de interfaces en la capa de control:**

En la capa de control se incorporaron interfaces como IUserController, IFoodController, IMenuController, INutritionInfoController e IStatisticsController. El objetivo de esta decisión fue desacoplar la definición de la responsabilidad de su implementación concreta. En términos prácticos, esto permite que cada controlador cumpla un contrato claro y que, en una futura evolución del sistema, pueda modificarse una implementación sin obligar a rehacer toda la estructura. Además, esta decisión mejora la legibilidad del proyecto, porque deja explícito qué funciones debe cumplir cada módulo.

- **Persistencia local en archivos CSV:**

Debido a que la primera iteración del proyecto debía trabajar con archivos y no con una base de datos, se optó por utilizar archivos CSV como mecanismo de almacenamiento. Esta decisión simplifica la carga, lectura y validación de la información, y además mantiene la solución alineada con lo solicitado en el enunciado. El uso de CSV también permite revisar fácilmente el contenido de los registros desde fuera del sistema, algo útil tanto para pruebas como para validación académica de los datos utilizados en la entrega.

- **Manejador genérico de persistencia:**

Para no repetir la misma lógica de lectura y escritura en diferentes módulos, se implementó la clase `FileHandler<T>`. Esta decisión permite reutilizar una estructura común para distintos tipos de datos, manteniendo una solución más uniforme y reduciendo código repetido. En

lugar de crear una clase independiente para cada archivo, el sistema aprovecha un mismo manejador adaptable a diferentes modelos, lo cual contribuye a una implementación más limpia y mantenible.

- **Centralización de la configuración:**

Las rutas de los archivos de usuarios, alimentos y menús se agruparon en ConfigurationItems.cs. Esta decisión busca evitar valores dispersos dentro de formularios o controladores, algo que haría más difícil localizar cambios y mantener consistencia. Al centralizar la configuración, el sistema gana mayor orden interno y facilita la administración de elementos compartidos, ya que las referencias principales quedan concentradas en un solo punto del proyecto.

- **Pantallas especializadas en lugar de una vista única:**

Cada módulo del sistema se resolvió mediante una ventana específica, en lugar de concentrar todo en una sola interfaz. Esta decisión se tomó para que la navegación fuera más clara, el flujo de uso más entendible y la interfaz más organizada. También ayuda a respetar el principio de responsabilidad única en la capa visual, ya que cada pantalla atiende una función concreta: registro, consulta, perfil, información nutricional o estadísticas. Desde la perspectiva del usuario, esto hace que el sistema resulte más intuitivo y menos recargado.

- **Validaciones básicas antes de guardar:**

En los módulos de usuarios, alimentos y menús se incorporaron validaciones orientadas a prevenir registros vacíos, datos inválidos y duplicados evidentes. Esta decisión se tomó para proteger la consistencia de la información y evitar que el sistema almacene datos incorrectos desde el inicio. Aunque se trata de validaciones básicas, cumplen una función importante dentro de la calidad general del proyecto, ya que mejoran la confiabilidad de los registros y reducen errores de uso.

- **Enfoque práctico en la primera iteración:**

Considerando que esta fase del proyecto se centra en funcionalidad, estructura y cumplimiento técnico más que en una interfaz altamente estilizada, se priorizó la claridad del flujo, la correcta separación de módulos y la estabilidad de las interacciones principales. Esta decisión permitió concentrar el esfuerzo en la lógica del sistema, la persistencia de datos y la organización del código, dejando una base más firme para futuras mejoras visuales o tecnológicas.

Estas decisiones buscan que el proyecto no solo funcione correctamente, sino que también mantenga una estructura comprensible, ordenada y defendible desde el punto de vista académico, además de dejar una base suficientemente flexible para una evolución posterior del sistema.

5. Technology Stack y entorno de ejecución

6. Arquitectura, patrón aplicado y estructura del proyecto

La solución Vitta fue construida siguiendo la arquitectura **Modelo-Vista-Controlador (MVC)**, la cual permite separar la representación de los datos, la lógica de control y la interacción con el usuario. Esta decisión se encuentra alineada con lo solicitado en el proyecto, ya que la aplicación debía utilizar Windows Forms en la capa visual y mantener una estructura organizada que facilitara futuras modificaciones.

Dentro de esta arquitectura, el **modelo** representa las entidades principales del sistema, la **vista** corresponde a los formularios con los que interactúa el usuario, y la **capa de control** se encarga de validar acciones, coordinar la lógica del negocio y gestionar el acceso a los datos. Gracias a esta separación, el proyecto resulta más comprensible, más mantenible y sencillo de revisar desde el punto de vista académico y técnico.

Modelo	Controlador	Vista
Clases como User, Food y Menu, encargadas de representar la información base del sistema mediante propiedades, constructores y estructuras de datos.	Clases como LoginController, UserController, FoodController, MenuController, NutritionInfoController y StatisticsController, responsables de coordinar validaciones, cálculos, consultas y acceso a la persistencia.	Formularios LoginView, RegisterView, DashboardView, ProfileView, FoodRegisterView, FoodConsultView, MenuRegisterView, NutritionInfoView y StatisticsView. Capturan acciones del usuario y muestran resultados.

Patrón o enfoque complementario utilizado

El repositorio de Vitta fue organizado para mantener separadas la documentación, los datos y el código fuente del sistema. En la raíz del proyecto se encuentran archivos principales como Vitta.sln, users.csv, foods.csv, menus.csv y README.md.

La carpeta **docs** guarda los documentos de apoyo y entrega del proyecto, como el entregable inicial, el archivo con los enlaces solicitados y la documentación final. La carpeta **src** contiene el código fuente de la aplicación y dentro de ella se agrupan los tres proyectos principales que forman la arquitectura MVC.

Dentro de src, la carpeta **VittaModel** contiene las entidades principales del sistema, como usuarios, alimentos y menús. La carpeta **VittaController** reúne la lógica de control, los controladores del sistema y también las subcarpetas **Abstractions**, donde están las interfaces y contratos, y **Extensions**, donde se ubican apoyos o utilidades adicionales. Por otro lado, la carpeta **VittaView** contiene los formularios de Windows Forms que conforman la interfaz gráfica, además de la subcarpeta **Configuration**, donde se centralizan elementos de configuración compartidos.

Esta forma de organizar el proyecto permite ubicar más fácilmente cada parte del sistema y entender mejor qué hace cada capa. También ayuda a que el código se vea más ordenado, más claro y fácil de revisar.

Además, esta estructura fue importante porque permitió trabajar el proyecto de una forma más organizada desde el inicio. Al tener separadas las carpetas y responsabilidades, fue más sencillo identificar dónde iba cada archivo, hacer cambios sin enredar otras partes del sistema y mantener una lógica más clara durante el desarrollo. Esto también deja una mejor base para seguir ampliando el proyecto después.

```
ProyectoVitta/
├── docs/
│   ├── Entregable1.pdf
│   ├── ProyectoVitta.txt
│   └── Documentacion_Final_Vitta.pdf
├── src/
│   ├── VittaModel/
│   │   ├── Food.cs
│   │   ├── Menu.cs
│   │   └── User.cs
│   ├── VittaController/
│   │   ├── Abstractions/
│   │   │   ├── IDataHandler.cs
│   │   │   ├── IFoodController.cs
│   │   │   ├── IMenuController.cs
│   │   │   ├── INutritionInfoController.cs
│   │   │   ├── IStatisticsController.cs
│   │   │   └── IUserController.cs
│   │   ├── Extensions/
│   │   │   └── ArgumentExtensions.cs
│   │   ├── FileHandler.cs
│   │   ├── FoodController.cs
│   │   ├── LoginController.cs
│   │   ├── MenuController.cs
│   │   ├── NutritionInfoController.cs
│   │   ├── StatisticsController.cs
│   │   └── UserController.cs
│   └── VittaView/
│       ├── Configuration/
│       │   └── ConfigurationItems.cs
│       ├── DashboardView.cs
│       ├── FoodConsultView.cs
│       ├── FoodRegisterView.cs
│       ├── LoginView.cs
│       ├── MenuRegisterView.cs
│       ├── NutritionInfoView.cs
│       ├── ProfileView.cs
│       ├── Program.cs
│       ├── RegisterView.cs
│       └── StatisticsView.cs
├── foods.csv
├── menus.csv
├── users.csv
├── README.md
└── Vitta.sln
```

7. Modelo de datos y persistencia en archivos CSV

La persistencia de Vitta se resolvió mediante tres archivos principales: users.csv, foods.csv y menus.csv. Esta decisión permitió manejar la información del sistema de una forma simple, ordenada y acorde con lo solicitado para esta primera iteración del proyecto. Cada archivo cumple una función específica dentro de la aplicación y permite separar los datos según el tipo de información que se necesita guardar.

Cuando el sistema inicia, la información de estos archivos es cargada en memoria para que los diferentes módulos puedan trabajar con ella. Posteriormente, cuando se registra, actualiza o elimina información, el sistema vuelve a escribir la lista completa en el archivo correspondiente. De esta manera, se mantiene una persistencia local sencilla, pero suficiente para el funcionamiento de la aplicación.

Archivo	Propósito	Campos principales	Volumen para entrega
users.csv	Almacena los usuarios registrados y los datos base necesarios para personalizar cálculos y consultas.	UserName, Password, Name, Weight, Height, Goal, ActivityLevel, DietType, Age, Sex	25 usuarios
foods.csv	Contiene el catálogo de alimentos con la información nutricional utilizada en menús, cálculos y estadísticas.	Name, Calories, Protein, Carbohydrates, Fat	84 alimentos
menus.csv	Guarda los menús diarios de cada usuario según la fecha y los tiempos de comida registrados.	UserName, MenuDate, Breakfast, MorningSnack, Lunch, AfternoonSnack, Dinner	2250 menús

La separación de los datos en estos tres archivos permite que el sistema mantenga una mejor organización interna. users.csv concentra la información personal del usuario, foods.csv reúne el catálogo nutricional de alimentos y menus.csv registra el consumo diario. Esta división hace más fácil entender qué guarda cada archivo, simplifica la lectura de los datos y evita mezclar información distinta en una sola fuente.

Observación de implementación:

En menus.csv, cada tiempo de comida se guarda como texto. Si en una misma comida hay varios alimentos, estos se separan con el símbolo |. Además, cada alimento se registra junto con su cantidad en un formato como Nombre del alimento xCantidad. Después, el sistema interpreta ese texto para identificar los alimentos y sus cantidades.

Esa información se relaciona con foods.csv para obtener calorías, proteínas, carbohidratos y grasas. Con este modelo, la persistencia en CSV fue suficiente para soportar el funcionamiento general de la primera iteración.

8. Desarrollo del sistema y explicación de módulos

El desarrollo de Vitta se realizó siguiendo una estructura modular, con el fin de que cada parte del sistema atendiera una responsabilidad específica y pudiera integrarse de manera ordenada con las demás. A nivel general, la aplicación fue construida para permitir que un usuario se registre, inicie sesión, gestione alimentos, construya menús diarios, consulte información nutricional y visualice estadísticas por fecha.

La implementación se apoyó en la separación entre modelo, control y vista. De esta forma, las entidades representan la información base del sistema, los controladores procesan validaciones, cálculos y operaciones sobre los archivos, y los formularios se encargan de la interacción con el usuario. Gracias a esta distribución, el proyecto mantiene una lógica de trabajo más clara y facilita la comprensión del funcionamiento general de cada módulo.

8.1 Pantallas y módulos principales

Pantalla / módulo	Descripción funcional	Capa principal
LoginView	Permite al usuario autenticarse y redirigirse al Dashboard. También habilita el acceso a la pantalla de registro.	Vista
RegisterView	Captura los datos del nuevo usuario, incluyendo objetivo, nivel de actividad, tipo de dieta, edad y sexo.	Vista
DashboardView	Funciona como menú principal y punto central de navegación a todos los módulos.	Vista
ProfileView	Muestra la información del usuario actual y permite actualizar sus datos.	Vista
FoodRegisterView	Permite agregar nuevos alimentos al catálogo nutricional.	Vista
FoodConsultView	Muestra la lista de alimentos, habilita búsqueda, edición y eliminación.	Vista
MenuRegisterView	Permite armar el menú diario por tiempos de comida, cargar menús previos, actualizarlos o eliminarlos.	Vista
NutritionInfoView	Resume calorías de mantenimiento, IMC, clasificación del IMC y distribución sugerida de macronutrientes.	Vista
StatisticsView	Consulta consumo diario o por período, calcula metas, días cumplidos y diferencias entre meta y consumo.	Vista

8.2 Controladores principales y su responsabilidad

Además de las vistas, el sistema se apoya en una capa de control que coordina la lógica de funcionamiento de cada módulo. Su papel principal es recibir la información de las pantallas, validarla, procesarla y relacionarla con los archivos de persistencia.

- **LoginController:** centraliza operaciones de inicio de sesión, registro y actualización delegada al controlador de usuarios.
- **UserController:** valida usuarios, comprueba duplicados y persiste cambios en users.csv.
- **FoodController:** administra el catálogo nutricional y permite búsquedas por nombre.
- **MenuController:** registra, consulta, actualiza y elimina menús por usuario y fecha.
- **NutritionInfoController:** calcula calorías de mantenimiento, IMC y distribución de macronutrientes.
- **StatisticsController:** interpreta el contenido de los menús y construye resúmenes diarios o por rango de fechas.

8.3 Relación general entre módulos

El funcionamiento del sistema sigue una secuencia lógica. Primero, el usuario se registra o inicia sesión; después puede actualizar su perfil, administrar el catálogo de alimentos y registrar sus menús diarios. A partir de esos datos, el sistema genera información nutricional y estadísticas por fecha.

Esta relación entre módulos permite que Vitta funcione como una solución conectada, donde la información registrada en una parte del sistema sirve de base para otras funcionalidades. Gracias a ello, el proyecto mantiene coherencia interna y logra integrar acceso, gestión, cálculo y análisis dentro de una misma aplicación.

9. Manual de instalación, ejecución e interacciones

Esta sección resume la forma en que se ejecuta y se utiliza Vitta dentro de su primera iteración. Su objetivo es mostrar, de manera práctica, qué se necesita para abrir el proyecto, cuál es el orden recomendado de uso y cuáles son las principales interacciones que puede realizar el usuario dentro del sistema. De esta manera, el manual complementa la documentación técnica con una guía breve y visual del funcionamiento general de la aplicación.

9.1 Requisitos previos

Para ejecutar correctamente el sistema se requiere lo siguiente:

Elemento	Descripción
Entorno de desarrollo	Visual Studio 2022 o una versión compatible con proyectos de Windows Forms
Solución principal	Archivo Vitta.sln
Archivos de datos	Acceso a los archivos CSV del proyecto: users.csv, foods.csv y menus.csv.
Configuración	Verificar las rutas definidas en ConfigurationItems.cs
Dependencias	Restauración automática de paquetes al compilar el proyecto

9.2 Pasos para ejecutar el sistema

1. Abrir el archivo Vitta.sln desde Visual Studio.
2. Verificar la ubicación de los archivos CSV en la clase ConfigurationItems.cs. Si el proyecto se mueve de carpeta o de computadora, actualizar esas rutas para que apunten a users.csv, foods.csv y menus.csv.
3. Configurar VittaView como proyecto de inicio.
4. Compilar la solución para restaurar dependencias y validar que no existan errores de compilación.
5. Ejecutar la aplicación y acceder con un usuario existente o registrar uno nuevo.

9.3 Flujo Recomendado

FLUJO VISUAL

El flujo de uso de Vitta sigue una secuencia simple y ordenada, donde el usuario inicia sesión o se registra, accede al panel principal y desde ahí navega hacia los diferentes módulos del sistema.

A continuación, se presenta el recorrido visual recomendado con base en las pantallas implementadas en el proyecto.

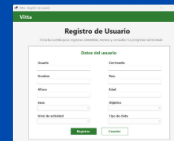


PASO 1. INICIO DE SESIÓN

La aplicación inicia en la pantalla LoginView, donde el usuario puede ingresar sus credenciales para autenticarse en el sistema. Desde esta misma ventana también se habilita el acceso al registro de un nuevo usuario.

PASO 2. REGISTRO DE USUARIO

Si el usuario no tiene una cuenta creada, puede registrarse desde RegisterView. En esta pantalla se capturan los datos personales y nutricionales básicos necesarios para el funcionamiento del sistema, como usuario, contraseña, nombre, peso, altura, edad, sexo, objetivo, nivel de actividad y tipo de dieta.



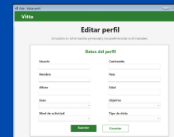
PASO 3. ACCESO AL PANEL PRINCIPAL

Una vez autenticado, el sistema dirige al usuario a DashboardView, que funciona como panel principal de navegación. Desde esta pantalla se puede acceder a los módulos de perfil, alimentos, menús, información nutricional y estadísticas.



PASO 4. ACTUALIZACIÓN DEL PERFIL

Desde ProfileView, el usuario puede revisar y modificar su información personal y nutricional. Esta pantalla permite actualizar datos importantes para los cálculos del sistema, como peso, altura, edad, objetivo, nivel de actividad y tipo de dieta.



PASO 5. CONSULTA Y GESTIÓN DE ALIMENTOS

En FoodConsultView, el usuario puede visualizar el catálogo de alimentos registrados, realizar búsquedas por nombre, editar registros existentes y eliminar alimentos cuando sea necesario. Este módulo sirve como base para la construcción de menús y cálculos nutricionales.

CONSULTA DE ESTADÍSTICAS NUTRICIONALES

Desde StatisticsView, el usuario puede revisar resultados diarios o por rango de fechas. En esta pantalla se muestran calorías consumidas, metas nutricionales, diferencias entre consumo y objetivo, además de resúmenes por período.



9.4 Interacciones importantes del sistema

Interacción	Qué realiza el sistema	Resultado visible
Registro de usuario	Valida campos obligatorios y luego guarda el usuario.	Nuevo registro persistido.
Alta de alimento	Guarda nombre y macronutrientes del alimento.	Catálogo ampliado.
Búsqueda de alimento	Filtra alimentos por nombre.	Lista reducida para consulta.
Guardado de menú	Asocia usuario, fecha y tiempos de comida con alimentos y cantidades.	Nuevo menú disponible.
Carga por fecha	Recupera el menú del usuario en una fecha específica.	Formulario rellenado.
Cálculo nutricional	Obtiene calorías, IMC y macros con base en datos del usuario.	Resumen inmediato.
Consulta estadística	Recorre los menús y suma nutrición por fecha o período.	Totales, promedios y diferencias.

9.5 Cálculos principales implementados

Calorías de mantenimiento: El sistema calcula una meta calórica diaria multiplicando el peso del usuario por un factor según su nivel de actividad. Los factores documentados en Vitta son: 30 para sedentario, 35 para ligero, 40 para moderado y 45 para alto.

Fórmula: $\text{Calorías de mantenimiento} = \text{Peso} \times \text{Factor de actividad}$

Índice de masa corporal (IMC): El IMC se obtiene dividiendo el peso entre la altura al cuadrado. Si la altura fue registrada en centímetros, el sistema primero la convierte a metros antes de aplicar la fórmula.

Fórmula: $\text{IMC} = \text{Peso} / (\text{Altura} \times \text{Altura})$

Clasificación del IMC: Una vez calculado el IMC, el sistema muestra una clasificación general según rangos como bajo peso, normal, sobrepeso u obesidad.

Distribución de macronutrientes: El sistema genera una distribución referencial de proteínas, carbohidratos y grasas según el objetivo y el tipo de dieta del usuario. En la documentación que llevas, por ejemplo, la dieta keto se describe con una distribución de 30% proteínas, 10% carbohidratos y 60% grasas.

Consumo y metas por fechas: En estadísticas, el sistema lee los alimentos registrados en menus.csv, identifica cada alimento y su cantidad, y luego lo relaciona con foods.csv para obtener calorías, proteínas, carbohidratos y grasas.

10. Resultados obtenidos y validación del proyecto

La validación técnica del proyecto se realizó revisando la organización general de la solución, la distribución de responsabilidades y la coherencia entre módulos, controladores, vistas y persistencia. A partir de esta revisión, se identifican los siguientes puntos de cumplimiento:

- La solución se encuentra separada en tres proyectos principales: modelo, controlador y vista.
- Las entidades base del dominio fueron definidas de manera explícita mediante clases como User, Food y Menu.
- La lógica principal del sistema se concentra en controladores especializados, evitando cargar excesivamente los formularios.
- El acceso a archivos se apoya en una implementación reutilizable mediante `FileHandler<T>`.
- La interfaz se desarrolla con Windows Forms, tal como lo solicita el proyecto.
- La solución incorpora revisión de buenas prácticas mediante el paquete SonarAnalyzer.CSharp.

En conjunto, estos elementos evidencian que el proyecto no fue construido únicamente para funcionar, sino también para mantener una estructura técnica clara y coherente con lo solicitado en la primera iteración.

10.1 Validación del conjunto de datos para entrega

Para fortalecer la entrega final, se trabajó con un conjunto de datos más amplio y consistente, con el fin de que las funcionalidades de consulta, cálculo y estadísticas tuvieran una base suficiente y más realista. Los resultados obtenidos fueron los siguientes:

Criterio	Resultado	Comentario
Usuarios registrados	25	Cumple mínimo y coincide con login y perfil.
Alimentos disponibles	84	Supera el mínimo y usa referencias cercanas al contexto costarricense.
Registros de menú	2250	Suficiente para estadísticas por fecha y periodo.
Días por usuario	90 - 90	Cada usuario tiene 90 días completos registrados.
Cobertura temporal	3 meses	Muestra continuidad y evita evidencia aislada.

11. Aprendizajes del desarrollo y Conclusiones

Uno de los principales aprendizajes del proyecto fue comprender que una aplicación pequeña también necesita organización desde el inicio. Separar entidades, controladores y vistas simplificó tanto la implementación como la corrección de errores, y evitó que la interfaz se convirtiera en el lugar donde ocurre toda la lógica.

También se reforzó la importancia de planificar antes de programar. El entregable inicial permitió descomponer el trabajo en funcionalidades, PBIs y tareas, lo que facilitó distribuir el avance y entender mejor el alcance real del sistema.

Desde el punto de vista técnico, el proyecto ayudó a practicar lectura y escritura de archivos, validaciones de entradas, trabajo con fechas, cálculo de métricas nutricionales y navegación entre ventanas de Windows Forms. Además, el uso de SonarAnalyzer hizo visible que la calidad del código no depende solo de que compile, sino también de cómo está estructurado.

Vitta cumple con el propósito de esta primera iteración al ofrecer un sistema funcional de gestión nutricional desarrollado en C# y Windows Forms, con una estructura organizada y coherente con la arquitectura MVC solicitada.

La documentación técnica demuestra cómo se tomaron las decisiones de diseño, cómo está compuesto el proyecto, cómo se ejecuta y de qué forma interactúan entre sí sus módulos. Esto fortalece la defensa del trabajo porque el sistema no se presenta únicamente como código, sino como una solución comprensible y justificable.

Finalmente, la ampliación del conjunto de datos a tres meses por usuario mejora la calidad de la evidencia del proyecto, ya que las consultas y estadísticas dejan de depender de pocos registros aislados. Con ello, el sistema se presenta no solo como un prototipo funcional, sino como una base seria para una segunda iteración más robusta.

12. Anexos Bibliografía

Repositorio del proyecto: <https://github.com/Nat-92CR/ProyectoVitta.git>

Herramienta de seguimiento: <https://trello.com/b/3z5MMCwt/proyecto-sistema-de-gestion-nutricional-vitta>

Archivos base de la solución: Vitta.sln, src/VittaModel, src/VittaController, src/VittaView, users.csv, foods.csv y menus.csv.

Nota de entrega: Este documento debe guardarse dentro de la carpeta docs del repositorio y acompañarse del archivo TXT con los enlaces solicitados, tal como lo establece el proyecto.